How Reliable Is Learned Cardinality Estimation?
A Data Model-Centric Analysis and Empirical Study

Shuxun Zhang[0000−0001−6517−9791] and Jiaheng Lu ✉[0000−0003−2067−454X]

Department of Computer Science, University of Helsinki, Helsinki, Finland
{shuxun.zhang,jiaheng.lu}@helsinki.fi

Abstract. Learning-based cardinality estimation (CE) has emerged as a promising alternative to statistics-based techniques. Yet its reliability remains an open question: directional bias, upper-tail risk, and degradation under data or workload shifts. Existing works typically organize learned CE by learning paradigm, such as query-driven versus data-driven, which can obscure how the underlying data model constrains estimator design and reliability. We adopt a data model-centric perspective, introducing a unified analytical framework spanning query encoding, data feature extraction, and model adaptation. We argue that the most informative classification axis shifts across data models: primary modeling target for relational CE, topology-handling strategy for graph CE, and cross-model integration strategy for multi-model CE. A relational case study grounds this analysis: a topmost-join probe of PostgreSQL reveals systematic underestimation; a five-seed evaluation with checkpoint reuse across workloads characterizes MSCN's workload-shift degradation; and a matched JOB-light comparison shows that learned estimation can substantially reduce, but does not eliminate, typical and tail errors. We further assess what current benchmarks reveal about reliability and identify open challenges toward robust, transparent, and reliable cardinality estimation.

Keywords: Cardinality Estimation · Learned Query Optimization · Multi-Model Databases · Reliability

1 Introduction

Cardinality estimation (CE) predicts the output size of query operators and intermediate subplans. In a cost-based query optimizer, these estimates are passed to the cost model during plan enumeration, helping the optimizer compare candidate execution plans and select one for execution. Although these plans are intended to be semantically equivalent, their execution costs may differ by several orders of magnitude. Inaccurate cardinalities can propagate through intermediate results, distort cost estimates, and lead to substantially slower plan choices without changing the logical query result. On the Join Order Benchmark (JOB), estimators in widely used systems have been shown to produce errors that grow rapidly with query complexity, and a decade later this remains a primary source of poor plans [11,12]. CE is not only an estimation-accuracy problem but also a critical factor in ensuring reliable database performance.

Classical cardinality estimators rely on simplifying assumptions, particularly uniformity within statistical summaries and independence among attributes or predicates. These assumptions become unreliable in the presence of skew, correlations, and complex joins [11,8]. Learning-based CE seeks to reduce this dependence on hand-crafted assumptions by either modeling data distributions or learning a direct mapping from queries to cardinalities [24,8]. Such methods have often achieved lower estimation errors on in-distribution workloads, but improved average accuracy itself does not ensure reliability. As learned estimators are fitted to a particular data distribution, query workload, or both, low median errors can coexist with severe error tails under data and workload shifts [28]. Evaluating learned CE requires examining not only average accuracy, but also systematic bias, tail behavior, and robustness under data and workload shifts.

Prior work has characterized the design space of learned CE or empirically evaluated its accuracy, robustness, and system impact, primarily in relational settings [8,24,28]. Broader studies treat CE as one component of learned query optimization [37], while graph-oriented analysis covers it within the scope of graph data management [26]. To our knowledge, no existing work jointly examines how the underlying data model reshapes learned CE design and reliability across relational, graph, and multi-model settings.

We argue that two properties help explain these differences. Data-side modelability concerns how well CE-relevant regularities can be represented compactly and queried efficiently. Query-side structural complexity concerns how much of a query's native structure and dependencies an estimator must preserve. We use these properties to explain how the data model shapes three recurring design dimensions: query encoding, data feature extraction, and model adaptation.

For relational CE, the primary modeling target provides the most informative distinction, separating distribution learning from query regression [8,24]. Graph CE, by contrast, must account for topological dependencies in query patterns, particularly those introduced by cycles [3]. For subgraph-matching queries, counting isomorphic embeddings is #P-hard [23]. Graph methods are more naturally distinguished by topology-handling strategies: direct structural encoding, path-sequence encoding, and substructure decomposition. Cross-model queries further combine heterogeneous query semantics, data summaries, and operator interactions. Because evidence for multi-model CE remains limited [33,4], we organize this setting by its cross-model integration strategy, which concerns how model-specific representations are reconciled and cross-model dependencies are captured. The observed strategies serve as a descriptive lens rather than a settled taxonomy. We conclude by extending the reliability perspective to pretrained and zero-shot CE, LLM-generated workloads, and semantic operators.

This paper makes three contributions. First, we introduce a data model-centric framework spanning query encoding, data feature extraction, and model adaptation. Second, we review representative methods, design strategies, and their reliability trade-offs across relational, graph, and multi-model CE. Third, we re-examine PostgreSQL bias and MSCN workload-shift behavior and assess what current benchmarks reveal about reliability.

2 Problem Setting and Analytical Framework

Given a query or intermediate expression q over a data instance D, cardinality estimation predicts the size of its result. Let $c(q, D) = |q(D)|$ denote the true cardinality under the semantics of the underlying data model, such as the number of rows in a relational result or matches of a graph pattern. For $c = c(q, D) > 0$ and $\hat{c} > 0$, estimation quality is commonly measured by the q-error,

$$\mathrm{qerr}(q) = \max\left\{\frac{\hat{c}}{c}, \frac{c}{\hat{c}}\right\}. \tag{1}$$

This metric treats over- and underestimation symmetrically and supports comparison across cardinality scales. Queries with $c = 0$ are excluded from our case study summaries as they require a separate convention. Estimated cardinalities are additionally bounded below by one before computing q-error.

Throughout this work, we use reliability to mean the stability of an estimator's error distribution along three dimensions: directional bias, upper-tail risk, and degradation under data or workload shift. We operationalize these dimensions using the underestimation rate, upper q-error percentiles, and changes in these statistics across data or workload conditions. Q-error measures estimation accuracy but does not by itself determine plan quality [11,16].

Two qualitative properties help interpret why the viable CE design space changes across data models. Data-side modelability is higher when the distributions, correlations, and structural dependencies relevant to cardinality can be represented compactly and used efficiently during estimation. Query-side structural complexity is higher when estimation must retain more native query topology, dependencies, and operator interactions. These are qualitative descriptors rather than measured scores. When relevant data regularities can be modeled compactly, and query structure is relatively simple, both distribution learning and query regression may be practical. As data dependencies become harder to model or query structures more complex, estimators must decide more explicitly what information to retain and how to encode, simplify, or decompose it.

Figure 1 summarizes three dimensions along which these properties shape estimator design. *Query encoding* concerns how predicates, operators, joins, paths, and higher-level structures such as join graphs or subgraph patterns become model inputs. *Data feature extraction* concerns which catalog metadata, statistical summaries, and structural features such as degree or path statistics are exposed to the estimator. *Model adaptation* concerns how a learning backbone, such as an MLP, GNN, or Transformer, is specialized to the data model through aggregation, neighborhood processing, and set or path representations. These dimensions are analytical rather than sequential, and they interact in practice. The following sections apply them to relational, graph, and multi-model CE.

3 Relational Cardinality Estimation

The relational model provides well-defined attribute-, predicate-, and join-level abstractions, together with mature statistical summaries such as histograms and

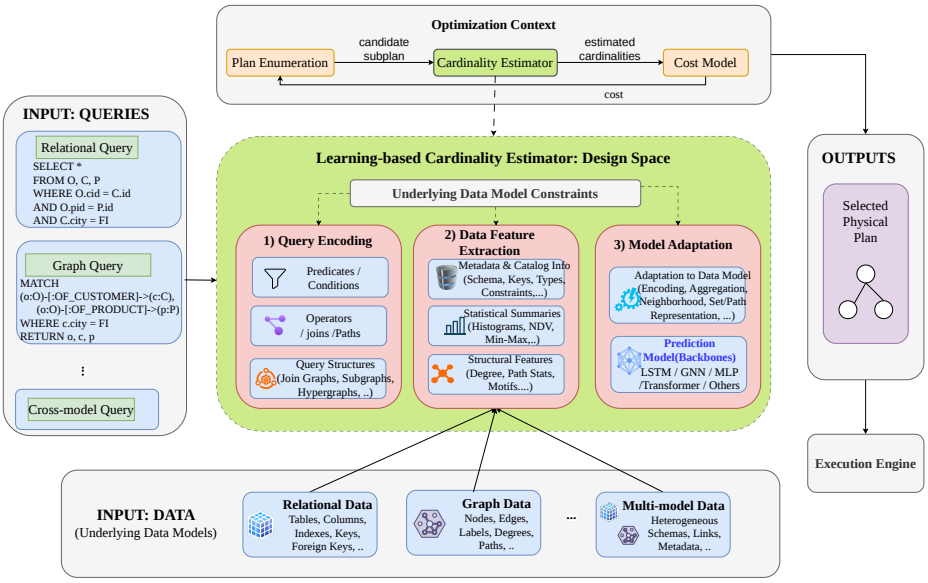


Fig. 1. A data model-centric view of learning-based cardinality estimation

distinct-value counts. Compared with graph-pattern CE, relevant data regularities and query structures are more readily expressed through these representations. This has supported two established design routes: distribution learning, which models data distributions and estimates cardinalities by probabilistic inference; and query regression, which predicts cardinalities directly from encoded queries [24,8]. Table 1 summarizes representative methods.

3.1 Distribution Learning

Distribution-learning methods construct probabilistic representations of the data and derive cardinalities by evaluating the probability mass a query selects. Their central trade-off is retaining the dependencies a query exercises while keeping model construction, maintenance, and inference affordable, and they differ in how the distribution is represented, decomposed, and queried.

NeuroCard uses a schema-wide autoregressive density model over the full outer join, capturing inter-column and inter-table correlations without independence assumptions and answering range queries through progressive sampling; it supports queries over table subsets, though its core formulation assumes an acyclic join schema and large attribute domains can increase model size [31].

DeepDB and FLAT take a probabilistic-circuit route: DeepDB uses relational sum-product networks (RSPNs) that recursively cluster rows and decompose weakly dependent columns for efficient, sampling-free inference, while FLAT's factorize-split-sum-product network (FSPN) adaptively combines conditional

Table 1. Representative learned relational CE methods.

Method	Primary design	Representation and mechanism	Characteristic limitation
Distribution learning			
NeuroCard [31]	Autoregressive density	Schema-wide join distribution; progressive sampling	Sampling cost; acyclic schemas
DeepDB [7]	Probabilistic circuit	RSPNs over row/column partitions; sampling-free inference	Depends on learned decomposition
FACE [27]	Normalizing flow	Invertible flow; pointwise density	Range needs Monte Carlo integration
FactorJoin [30]	Query-time composition	Per-table conditional distributions; query-time factor graph	Binning and bound approximations
Query regression			
MSCN [10]	Flat-set encoding	Sets of tables, joins, predicates; pooling	No explicit join-graph topology
JGMP [21]	Join-graph encoding	Join graph; message passing	Restricted to supported join forms
ALECE [14]	Data-aware regression	Query–histogram cross-attention	Requires maintained histograms
GRASP [29]	Data-agnostic composition	Per-table primitives; count sketch	Join-key independence; sketch approximation

and independent factorization by attribute correlation, balancing dependency representation, model size, and probability-computation efficiency [7,38].

FACE avoids column-wise decomposition by using an invertible normalizing flow that yields explicit pointwise densities, though range cardinalities are approximated by Monte Carlo integration [27].

A second group shifts cross-table reasoning to query time rather than learning one joined distribution: FactorJoin builds a query-specific factor graph over per-table conditional distributions supplied by learned or conventional single-table estimators, combining them through approximate inference with join-key binning and probabilistic bounds [30], while ASM couples autoregressive per-table statistics with sampling-based and multi-dimensional statistics merging to estimate many subqueries without assuming join-key independence [9]. We classify these hybrid methods by their primary estimation route and therefore group both under distribution learning despite their query-time composition. Across these methods, reliability depends not only on the quality of the un-

derlying data representations, but also on whether factorization, sampling, and cross-table composition preserve the dependencies exercised by a query.

3.2 Query Regression

Query-regression methods learn a predictor from labeled queries to cardinalities; their primary target is the query-to-cardinality mapping, even when auxiliary data signals are incorporated. This line progresses from flat encodings toward structured and data-aware representations.

MSCN represents tables, join predicates, and filters as three unordered sets, applies shared transformations and average pooling within each, and regresses from their concatenation [10]. This accommodates variable-size queries, but independently pooling join predicates can render structurally distinct join graphs indistinguishable. JGMP addresses this ambiguity by encoding queries as join graphs and applying message passing, thereby preserving structural associations discarded by flat pooling; the same work proposes an orthogonal relational-algebra-based self-supervised regularizer to improve sample efficiency [21].

ALECE moves in a complementary direction, using cross-attention to condition query representations on per-attribute histogram states, thereby capturing interactions between query predicates and the current data distribution [14].

Robust-MSCN masks query features during training and combines the remaining representation with DBMS estimates and join bitmaps, reducing reliance on workload-specific query features under drift [17]. At the boundary of this route, GRASP adopts a data-agnostic compositional design, learning per-table cardinality and join-key models from queries to compose estimates for unseen or underrepresented join templates [29]. Robust-MSCN and GRASP illustrate contrasting responses to generalization: one anchors predictions in updated data-side signals, whereas the other avoids data access.

A central reliability question for query-driven methods is generalization: how estimates behave beyond the query structures and data conditions represented during training. Our case study focuses on workload shift for MSCN.

Whereas relational methods divide by their primary modeling target, graph CE shifts the organizing axis to how query topology is encoded or decomposed.

4 Graph Cardinality Estimation

Graph CE spans query semantics, including conjunctive queries over knowledge graphs, path patterns, and subgraph counting under homomorphism or isomorphism semantics. Across these settings, topology is a first-class determinant of cardinality: connectivity, branching, cycles, and path constraints induce dependencies that are difficult to capture using attribute-level summaries alone. This increases query-side structural complexity and shifts data-side features toward degree, path, label, motif, and neighborhood statistics. We therefore organize learned graph estimators by their dominant topology-handling strategy: direct

Table 2. Representative learned graph CE methods, with a non-learned contrast.

Method	Query setting	Representation and mechanism	Characteristic limitation
Direct structural encoding			
GNCE [22]	Conjunctive SPARQL, knowledge graphs	RDF2Vec + occurrence features; directed message passing on query graph	Higher errors on complex query shapes
NeurSC [25]	Subgraph counting, undirected vertex-labeled	Query + candidate substructures; intra-/ inter-graph networks; Wasserstein-guided training	Candidate extraction and interaction overhead
FlowSC [5]	Subgraph counting, undirected vertex-labeled	Filtered candidate space; one-pass bottom-up message passing emulates candidate-tree counting	Sensitive to limited training coverage
Path-sequence encoding			
SPACE [1]	Path patterns, directed labeled graphs	Label/edge-type sequence; cardinality-based encoding	Restricted to path-pattern queries
Substructure decomposition			
ALSS [36]	Subgraph counting, homomorphism / isomorphism	Depth-ℓ BFS trees; GNN + self-attention + MLP; active learner	Exact-count labeling for adaptation
Non-learned contrast			
FaSTest [23]	Subgraph counting	Candidate filtering; adaptive tree + stratified sampling	Sampling failure on large sample spaces

structural encoding, path-sequence encoding, or substructure decomposition. Because these strategies target different graph-query semantics, their reported errors are not directly comparable. Table 2 summarizes representative methods.

4.1 Direct Structural Encoding

Direct structural encoders retain a graph-structured representation throughout estimation, with varying degrees of query-specific data involvement.

GNCE targets conjunctive SPARQL queries over RDF knowledge graphs. It initializes bound query atoms with RDF2Vec embeddings and occurrence frequencies, assigns explicit representations to variables, and applies direction-aware message passing over the query graph before graph-level cardinality regression [22]. GNCE avoids fixed independence and uniformity assumptions and

generalizes to entities unseen in the training workload, although its reported errors increase on structurally more complex query shapes.

NeurSC incorporates query-specific data context for subgraph counting. It extracts candidate substructures from the data graph and jointly encodes them with the query using intra- and inter-graph networks to capture structure within and across the graphs; a Wasserstein discriminator guides representation learning during training [25]. This additional context grounds the estimate in data-side structure, but candidate extraction and query–candidate interactions can make inference costly on large graphs. The formulation is restricted to undirected, vertex-labeled graphs.

FlowSC uses task-specific message passing. It filters the candidate space and controls message direction, range, and rounds to emulate bottom-up candidate-tree counting [5]. It is more stable on complex queries but remains sensitive to sparse training coverage for large queries.

4.2 Path-Sequence Encoding

Path-sequence encoding provides a topology-aligned representation for intrinsically linear queries. SPACE represents path patterns as a sequence of node labels and edge types, and combines structural sequence encoding with a cardinality-aware encoding that integrates cardinality signals during learning [1]. Its evaluation reports lower q-error and training time than the baselines. The evaluated formulation is restricted to path patterns; branching or cyclic patterns would require additional choices for representing traversal order and non-path edges.

4.3 Substructure Decomposition

Unlike the previous two strategies, substructure decomposition explicitly splits the query graph into smaller units and learns how to combine their representations. ALSS follows this design for subgraph counting under homomorphism or subgraph-isomorphism semantics [36]. Its learned sketch decomposes an n-node query graph into n overlapping substructures, using a breadth-first search tree of depth ℓ rooted at each query vertex. It encodes each tree with a graph neural network, assigns learned self-attention weights to the resulting representations, and feeds their weighted aggregation to a final MLP regressor.

This learned combiner distinguishes ALSS from earlier approximate estimators based on decomposition, which combine substructure counts under independence assumptions that can cause substantial estimation error. The learned weighting evaluates each substructure's query-specific importance, while the nonlinear regression avoids a fixed independence-based aggregation rule. The accompanying active learner selects uncertain queries from the arriving workload, obtains their exact counts, and updates the model by re-optimizing the loss on the enlarged training set. This supports adaptation as the workload changes, but exact labeling is costly; the authors therefore suggest periodic batch updates using true counts computed offline. Decomposition depth, workload coverage, and the availability of exact labels together set the limits of this design.

Learned versus sampling-based reliability. Within subgraph counting, FaSTest combines candidate filtering, adaptive tree sampling, and worst-case-optimal stratified sampling, reporting accuracy gains of up to three orders of magnitude over the GNN-based baselines considered [23]. The FlowSC evaluation shows that this advantage is workload-dependent: FaSTest attains a q-error near 1 on HPRD, but exceeds 10^9 on 20-vertex WordNet queries because large sample spaces cause frequent sampling failure [5]. On Yeast, FlowSC and NeurSC tend to overestimate small counts and underestimate large ones, whereas ALSS achieves a more balanced error distribution through active learning. In these evaluations, the dominant failure modes differ: sampling degrades as filtered sample spaces expand, whereas learned estimators depend on training coverage, optimization stability, and adaptation.

Across these strategies, query topology enters the estimator as an explicit graph, an ordered path, or overlapping local structures. Representation determines which structural dependencies remain available, while query semantics define the counting target. The reliability of a given estimate depends on how well the chosen representation, training workload, and sampling process cover it. Multi-model CE additionally requires cross-model integration to produce a common cardinality estimate.

5 Multi-Model Cardinality Estimation

Multi-model CE must combine model-specific representations while capturing dependencies across model boundaries, creating coupled challenges across all three dimensions of our framework. As work on learned multi-model CE remains sparse, we organize existing methods and the open design space by cross-model integration strategy. Integration may occur before final estimation through explicit transformation into a shared representation, within a jointly trained estimator, or after model-specific estimation by composing separate outputs.

Pre-estimation representation bridging. This strategy converts heterogeneous model-specific representations into a shared representation before final estimation. LVE represents hierarchical constraints through traversal orders and builds a joint kernel-density estimator for structural and value joins [33]. Qi et al. sample relational tuples and tree nodes at two levels and apply discrete learning to the resulting tree-node samples [20]. In both cases, the bridging logic is specified by the traversal or sampling procedure, after which estimation operates on the transformed representation. Reliability therefore depends on whether this transformation preserves the cross-model dependencies exercised by the query.

Joint learned estimation. The second strategy performs cross-model integration within a single jointly trained estimator, rather than through a separate transformation procedure or post-estimation composition. MANN encodes SQL and AQL queries using collection names, predicates, values, and logical operators, groups the resulting vectors by collection, processes each group with a collection-specific attention head, and pools the head outputs before cardinality regression [4]. Thus, collection-specific representations and their interactions are

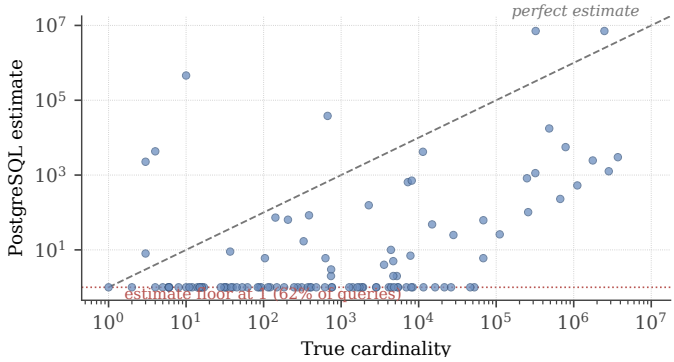


Fig. 2. PostgreSQL estimates versus true cardinalities on JOB.

learned jointly within one model. However, its shared encoding does not explicitly represent graph paths, edge directions, or document nesting. The formulation also excludes grouping, aggregation, and subqueries, while its evaluation is limited to static datasets and predominantly generated query workloads. MANN therefore represents an early joint query-regression approach rather than a complete treatment of model-native structures in cross-model queries.

Post-estimation composition. The third strategy would retain a separate estimator for each data model and combine their outputs after model-specific estimation. This preserves representations suited to each model and could reuse mature model-specific estimators. Its central challenge is that independently produced estimates do not determine the joint cardinality of a cross-model query unless dependencies across models are also captured. In the literature reviewed here, we found no established learned mechanism for modeling these dependencies. We therefore present post-estimation composition as a forward-looking route rather than an established method family.

6 Case Study: Reliability in Practice

Previous sections synthesize published evidence across data models; our empirical case study is relational, probing PostgreSQL bias and MSCN workload shift.

Directional bias in a native estimator. We evaluated all 113 JOB queries on PostgreSQL 18.1 with `default_statistics_target`=100 and statistics collected by `ANALYZE`, and no declared foreign-key constraints or extended statistics objects. For each query, we read the estimate from the `Plan Rows` value of the topmost join node in the `EXPLAIN (FORMAT JSON)` plan and obtained the true cardinality using an equivalent `COUNT(*)` rewrite. Because q-error is defined only for positive true cardinalities, we retain the 108 nonempty queries.[1]

Figure 2 shows strongly directional errors: 92.6% of the queries are underestimated, and 62% receive an estimate of exactly one row. This concentration

[1] The excluded queries are 2c, 5a, 5b, 10b, and 32a.

at the floor is consistent with PostgreSQL's default independence assumptions across multiple predicates and its clamping of nonzero row estimates to at least one. The median q-error is 261; only 17% of the queries fall within a factor of ten of the truth, the 95th percentile exceeds 14,000, and the worst case reaches 52,306. Grouped by join width, the median q-error is 57 for queries joining at most six tables ($n = 21$), 307 for queries joining seven to nine tables ($n = 50$), and 275 for queries joining at least ten tables ($n = 37$). The error rises roughly fivefold and then plateaus, though the widest group also spans the planner's GEQO threshold. This topmost-join probe recovers, more than a decade later, the systematic underestimation pattern long associated with JOB [11,12].

Workload shift in a learned estimator. We trained MSCN [10] in five independent runs using predefined random seeds 41–45, modifying the authors' implementation only to reuse the same trained checkpoint across all three workloads within each run. Each run followed the authors' recommended reproduction configuration. We evaluated each checkpoint on Synthetic, Scale, and JOB-light. Table 3 reports the mean and standard deviation across seeds.

Across five seeds, Synthetic yields a median q-error of 1.21 ± 0.01 and a 95th percentile of 8.10 ± 0.13. As previously observed [10], both shifts degrade MSCN, but our five-seed results distinguish their failure patterns. On Scale, which includes larger join counts, the median changes modestly to 1.46 ± 0.05, whereas the 95th percentile rises nearly eighteenfold to 144.72 ± 19.95, revealing severe tail deterioration. JOB-light raises typical error more visibly, with a median of 2.49 ± 0.27 and a 95th percentile of 63.77 ± 22.08. This ordering is consistent across seeds: JOB-light has the highest median, whereas Scale has the heaviest tail. Accuracy is therefore a property of the estimator–workload pair rather than of the estimator alone. Within each run, reusing one checkpoint holds the trained model fixed when comparing workloads; repeating the design across five seeds additionally measures sensitivity to model initialization.

The full-JOB and MSCN probes answer different questions, and their absolute q-errors are not directly comparable; the shared JOB-light workload, however, supports a direct comparison. On these 70 queries, PostgreSQL underestimated 69 and reached median and 95th-percentile q-errors of 15.39 and 1,600.33, compared with 2.49 ± 0.27 and 63.77 ± 22.08 for MSCN. On this matched workload, MSCN substantially improves both typical and tail accuracy, while its workload sensitivity and remaining tail errors indicate that reliability failures are reduced rather than eliminated. The comparison is not resource-matched: MSCN used 90,000 labeled training queries, whereas PostgreSQL used default statistics, so it does not establish lower deployment cost or better plans.

What current benchmarks measure. Our two probes expose different reliability dimensions, and similar limitations recur across evaluation practice. JOB targets optimizer failures on correlated multi-join queries [11], estimator-specific workloads probe generalization beyond their training distributions [10], and later benchmarks scale up workloads and add plan-quality metrics such as plan cost and end-to-end time [6,16]. Across these efforts, three properties persist: coverage is predominantly relational, with G-CARE providing a unified

Table 3. MSCN q-error across five random seeds (mean ± standard deviation; maximum reported as a range).

Workload	n per seed	Median	95th pct.	Max range
Synthetic (in-distribution)	5000	1.21 ± 0.01	8.10 ± 0.13	1703.50–2702.63
Scale (larger join counts)	500	1.46 ± 0.05	144.72 ± 19.95	3309.95–3852.91
JOB-light (JOB-derived)	70	2.49 ± 0.27	63.77 ± 22.08	490.47–647.44

framework for subgraph-matching CE [18] and multi-model evaluation relying on system-level benchmarks that do not isolate estimator behavior [34]; evaluation reports accuracy percentiles on fixed workloads, while controlled data or workload shifts remain uncommon [17,28]; and directional bias and tail risk are rarely reported as first-class quantities and may require re-analysis. Benchmark design therefore limits reliability claims, especially beyond the relational model.

7 Open Challenges and Future Work

We identify two immediate challenges and two emerging directions.

Reliability-oriented evaluation. Building on the limitations identified in Section 6, future protocols should treat directional bias, floor behavior, tail risk, run-to-run variation, and degradation under controlled shifts as first-class outcomes. Graph CE additionally requires explicit separation of query semantics and benchmark assumptions across estimators.

Adaptation without fresh ground truth. Many query-driven adaptation mechanisms require fresh cardinality labels. ALSS obtains exact counts for selected queries, while drift-aware relational methods reduce adaptation cost or sensitivity without eliminating deployment overhead [13,17]. When labels are unavailable, an alternative to adapting the estimator is to recognize when it should not be trusted. Pessimistic estimators offer a complementary, non-learned notion of reliability here: methods such as LpBound compute guaranteed upper bounds on join sizes, trading estimate tightness for a one-sided error guarantee [35]. Falling back to such bounds when a learned estimate cannot be trusted is a natural hybrid direction. The subgraph-counting comparison in Section 4 motivates selective fallback, since learned and sampling-based estimators fail in different regions. Using such a strategy safely requires calibrated uncertainty, a suitable fallback estimator, and an optimizer policy that accounts for estimation latency and plan impact.

Pretrained and zero-shot estimation. PRICE learns transferable low-level distribution and query features across multiple databases and applies them to unseen databases, optionally after fine-tuning [32]. CardBench complements this direction with query workloads and statistics across 20 databases; its zero-shot experiments report clearly lower accuracy on unseen datasets than instance-based training [2]. This shifts the reliability question toward cross-database transfer: zero-shot evaluation should vary schemas, distributions, and workloads

rather than report aggregate accuracy alone. Whether graph topology or cross-model structure admits similarly transferable representations remains open.

LLM-generated workloads and semantic operators. AI agents may become a major source of database workloads, issuing high-throughput, heterogeneous, and iterative queries as they explore and refine solutions [15]. This shift strengthens rather than removes the CE problem. LLMs may diversify training-query templates, but they do not eliminate the need for exact cardinality labels, and synthetically generated workloads may still differ from those produced by deployed agents. Their coverage and distribution therefore require validation. Semantic filters and joins pose a second CE problem: conventional statistics do not directly characterize natural language predicates whose outcomes depend on prompts, models, and execution policies [19]. Estimation may instead rely on samples, cached outcomes, operator semantics, and calibrated uncertainty.

8 Conclusion

This paper examined learned cardinality estimation through a data model-centric lens, arguing that the most informative classification axis shifts with the underlying data model. The relational case study exposed systematic underestimation in PostgreSQL and workload-dependent degradation in MSCN, while their matched JOB-light comparison showed that learned estimation can substantially reduce, but does not eliminate, typical and tail errors. Reliable estimation is a joint property of the estimator, workload, and data model, and evaluation practice should measure it accordingly.

Acknowledgments. An earlier, non-archival version of this work was presented as a tutorial at PAKDD 2026. This paper adds reliability framing, five-seed analysis, matched JOB-light comparison, and benchmark assessment. This work received partial support from the China Scholarship Council.

Disclosure of Interests. The authors declare they have no competing interests.

References

1. Aytimur, M., Chondrogiannis, T., Grossniklaus, M.: Space: cardinality estimation for path queries using cardinality-aware sequence-based learning. Proceedings of the ACM on Management of Data **3**(3), 1–26 (2025)
2. Chronis, Y., Wang, Y., Gan, Y., Abu-El-Haija, S., Lin, C., Binnig, C., Özcan, F.: Cardbench: A benchmark for learned cardinality estimation in relational databases. arXiv preprint arXiv:2408.16170 (2024)
3. Deeds, K., Sabale, D., Kayali, M., Suciu, D.: Color: A framework for applying graph coloring to subgraph cardinality estimation. Proceedings of the VLDB Endowment **18**(2), 130–143 (2024)
4. Deng, R., Wu, A.: MANN: Multi-head attention network model for estimating cardinality in multi-model databases. In: Proceedings of the 2024 7th International Conference on Computer Information Science and Artificial Intelligence. pp. 490–497 (2024)

5. Guo, Q., Yang, J., Zhang, W., Wang, H., Zhang, Y., Lin, X.: Efficient and accurate subgraph counting: A bottom-up flow-learning based approach. Proceedings of the VLDB Endowment **18**(8), 2695–2708 (2025)
6. Han, Y., Wu, Z., Wu, P., Zhu, R., Yang, J., Tan, L.W., Zeng, K., Cong, G., Qin, Y., Pfadler, A., et al.: Cardinality estimation in dbms: a comprehensive benchmark evaluation. Proceedings of the VLDB Endowment **15**(4), 752–765 (2021)
7. Hilprecht, B., Schmidt, A., Kulessa, M., Molina, A., Kersting, K., Binnig, C.: DeepDB: learn from data, not from queries! Proceedings of the VLDB Endowment **13**(7), 992–1005 (2020)
8. Kim, K., Jung, J., Seo, I., Han, W.S., Choi, K., Chong, J.: Learned cardinality estimation: An in-depth study. In: Proceedings of the 2022 international conference on management of data. pp. 1214–1227 (2022)
9. Kim, K., Lee, S., Kim, I., Han, W.S.: Asm: Harmonizing autoregressive model, sampling, and multi-dimensional statistics merging for cardinality estimation. Proceedings of the ACM on Management of Data **2**(1), 1–27 (2024)
10. Kipf, A., Kipf, T., Radke, B., Leis, V., Boncz, P., Kemper, A.: Learned cardinalities: Estimating correlated joins with deep learning. In: CIDR. www.cidrdb.org (2019)
11. Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., Neumann, T.: How good are query optimizers, really? Proceedings of the VLDB Endowment **9**(3), 204–215 (2015)
12. Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., Neumann, T.: Still asking: How good are query optimizers, really? Proceedings of the VLDB Endowment **18**(12), 5531–5536 (2025)
13. Li, B., Lu, Y., Kandula, S.: Warper: Efficiently adapting learned cardinality estimators to data and workload drifts. In: Proceedings of the 2022 International Conference on Management of Data. pp. 1920–1933 (2022)
14. Li, P., Wei, W., Zhu, R., Ding, B., Zhou, J., Lu, H.: ALECE: An attention-based learned cardinality estimator for SPJ queries on dynamic workloads. Proceedings of the VLDB Endowment **17**(2), 197–210 (2023)
15. Liu, S., Ponnapalli, S., Shankar, S., Zeighami, S., Zhu, A., Agarwal, S., Chen, R., Suwito, S., Yuan, S., Stoica, I., Zaharia, M., Cheung, A., Crooks, N., Gonzalez, J., Parameswaran, A.G.: Supporting our AI overlords: Redesigning data systems to be agent-first. In: Proceedings of the 16th Conference on Innovative Data Systems Research (CIDR) (2026)
16. Negi, P., Marcus, R., Kipf, A., Mao, H., Tatbul, N., Kraska, T., Alizadeh, M.: Flow-loss: learning cardinality estimates that matter. Proceedings of the VLDB Endowment **14**(11), 2019–2032 (2021)
17. Negi, P., Wu, Z., Kipf, A., Tatbul, N., Marcus, R., Madden, S., Kraska, T., Alizadeh, M.: Robust query driven cardinality estimation under changing workloads. Proceedings of the VLDB Endowment **16**(6), 1520–1533 (2023)
18. Park, Y., Ko, S., Bhowmick, S.S., Kim, K., Hong, K., Han, W.S.: G-CARE: A framework for performance benchmarking of cardinality estimation techniques for subgraph matching. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. pp. 1099–1114 (2020)
19. Patel, L., Jha, S., Pan, M., Gupta, H., Asawa, P., Guestrin, C., Zaharia, M.: Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS. Proceedings of the VLDB Endowment **18**(11), 4171–4184 (2025)
20. Qi, L., Jin, P., Wan, S.: Efficient selectivity estimation for relation-tree joins in multi-model databases. In: 2021 IEEE International Conference on Big Data (Big Data). pp. 5998–6002 (2021)

21. Reiner, S., Grossniklaus, M.: Sample-efficient cardinality estimation using geometric deep learning. Proceedings of the VLDB Endowment **17**(4), 740–752 (2023)
22. Schwabe, T., Acosta, M.: Cardinality estimation over knowledge graphs with embeddings and graph neural networks. Proceedings of the ACM on Management of Data **2**(1), 1–26 (2024)
23. Shin, W., Song, S., Park, K., Han, W.S.: Cardinality estimation of subgraph matching: A filtering-sampling approach. Proceedings of the VLDB Endowment **17**(7), 1697–1709 (2024)
24. Sun, J., Zhang, J., Sun, Z., Li, G., Tang, N.: Learned cardinality estimation: A design space exploration and a comparative evaluation. Proceedings of the VLDB Endowment **15**(1), 85–97 (2021)
25. Wang, H., Hu, R., Zhang, Y., Qin, L., Wang, W., Zhang, W.: Neural subgraph counting with wasserstein estimator. In: Proceedings of the 2022 International Conference on Management of Data. pp. 160–175 (2022)
26. Wang, H., Zhang, Y., Zhang, W.: Machine learning for graph data management and query processing. Proceedings of the VLDB Endowment **18**(12), 5499–5503 (2025)
27. Wang, J., Chai, C., Liu, J., Li, G.: Cardinality estimation using normalizing flow. The VLDB Journal **33**(2), 323–348 (2024)
28. Wang, X., Qu, C., Wu, W., Wang, J., Zhou, Q.: Are we ready for learned cardinality estimation? Proceedings of the VLDB Endowment **14**(9), 1640–1654 (2021)
29. Wu, P., Kang, R., Zhang, T., Chen, J., Marcus, R., Ives, Z.G.: Data-agnostic cardinality learning from imperfect workloads. Proceedings of the VLDB Endowment **18**(8), 2519–2532 (2025)
30. Wu, Z., Negi, P., Alizadeh, M., Kraska, T., Madden, S.: Factorjoin: a new cardinality estimation framework for join queries. Proceedings of the ACM on Management of Data **1**(1), 1–27 (2023)
31. Yang, Z., Kamsetty, A., Luan, S., Liang, E., Duan, Y., Chen, X., Stoica, I.: Neurocard: one cardinality estimator for all tables. Proceedings of the VLDB Endowment **14**(1), 61–73 (2020)
32. Zeng, T., Lan, J., Ma, J., Wei, W., Zhu, R., Zhou, Y., Li, P., Ding, B., Lian, D., Wei, Z., et al.: PRICE: A pretrained model for cross-database cardinality estimation. Proceedings of the VLDB Endowment **18**(3), 637–650 (2024)
33. Zhang, C., Lu, J.: Selectivity estimation for relation-tree joins. In: Proceedings of the 32nd International Conference on Scientific and Statistical Database Management. pp. 1–12 (2020)
34. Zhang, C., Lu, J., Xu, P., Chen, Y.: Unibench: a benchmark for multi-model database management systems. In: Technology conference on performance evaluation and benchmarking. pp. 7–23. Springer (2018)
35. Zhang, H., Mayer, C., Abo Khamis, M., Olteanu, D., Suciu, D.: Lpbound: Pessimistic cardinality estimation using ℓ_p-norms of degree sequences. Proceedings of the ACM on Management of Data **3**(3), 1–27 (2025)
36. Zhao, K., Yu, J.X., Li, Q., Zhang, H., Rong, Y.: Learned sketch for subgraph counting: a holistic approach. The VLDB Journal **32**(5), 937–962 (2023)
37. Zhu, R., Weng, L., Ding, B., Zhou, J.: Learned query optimizer: What is new and what is next. In: Companion of the 2024 International Conference on Management of Data. pp. 561–569 (2024)
38. Zhu, R., Wu, Z., Han, Y., Zeng, K., Pfadler, A., Qian, Z., Zhou, J., Cui, B.: FLAT: fast, lightweight and accurate method for cardinality estimation. Proceedings of the VLDB Endowment **14**(9), 1489–1502 (2021)